

Introduction to Performance Engineering

Performance engineering is a complex activity, which is based on existing, fundamental knowledge, but also requires active investigation.

The goal typically is to reduce the consumption of resources (be it time, compute time or energy - this is getting more and more important) for a given computational task.

This brief introductory courses aims to:

- give an overview of the knowledge required to do performance engineering work, in the spirit of reducing the *unknown unknowns* for beginners, with a focus on HPC and scientific computing;
- discuss the tools to analyze application behaviour in the most common programming languages used in HPC in the field of scientific research (profiling, tracing);
- give elementary notions of computer architecture, including CPU, memory systems, GPU and Filesystem, which sets the maximum performance that can be achieved;
- give a hands-on demonstration of the performance optimization loop.

The choice of topics is based on the sources of inspiration (see below) and on the personal experiences of the maintainers of this lesson, with a slight bias - which must be acknowledged and managed - towards *interesting* problems.

The closed loop of performance tuning

Before starting: ensuring correctness

Performance tuning requires to change the software in some way, and this might unadvertently break it.

Discussion

What do you think it is the most practical way to guarantee that the performance optimization work does not break the code?

How often would you check?

✓ Possible approaches (but let us discuss first)

The more frequent the testing, the less the work needed to find the cause of a bug, but the more the testing overhead.

For this reason, it would be ideal to have tests that:

- do not require manual inspection of the result (full automation)
- take very little time to run
- are as comprehensive as possible
- do not take much time to write and maintain

These aspects are obviously in contrast with each other: a reasonable balance must be struck, and it is typically down to personal preferences and experience.

In the absence of pre-existing tests, a common approach (typically used to ensure correctness) is to write *characterisation tests*:

1. save/determine the output of the legacy/reference/original code for a given input
2. use that as an oracle for the optimized code.

Knowledge of the algorithms and of their numerical properties helps with determining the most important test cases.

Using **version control** is also paramount, as with any activity related to software development.

Performance tuning workflow

Once a way to ascertain the correctness of the code is available (preferably *quick* and *automated*), performance optimization typically proceeds in an iterative fashion.

“Scientific” workflow:

1. Make a falsifiable hypothesis (why is my code slow?)
2. Get data and analyse it, plan code changes (design experiment)
3. Action: implement changes (perform experiment)

Another, more concrete, view of the same approach:

Phases of the Performance Tuning loop

Step	Possible problems	Problem mitigation (discuss)
1. Measurement	What to measure? And how?	Domain knowledge (which use cases are relevant?) Computer arch. knowledge Performance tool knowledge

Step	Possible problems	Problem mitigation (discuss)
2. Analysis	Performance analysis tools might produce <i>a lot</i> of data. What is relevant?	Computer arch. knowledge (Performance tool knowledge)
3 Generation of Alternatives	How to make hypotheses?	Computer arch. knowledge
4. Implementation	Is a code change worth it? (complexity vs. performance). Redundancy of optimizations	Version Control, Proper architecture, Domain knowledge (definition of relevant use cases)

It is also important:

- to keep track of the progress done during the iterations of the performance optimization loop and of the information (or lack thereof) that have influenced any decision. This can be done, for example, with a [logbook](#).
- to strive for performance reproducibility, keeping all [sources of performance irreproducibility](#) under control

Porting to accelerators, Parallelization and performance engineering

In order for software to run efficiently modern HPC clusters, typically it needs to be capable of:

- using multiple cores on the same host (shared memory)
- using multiple nodes (distributed memory)
- using accelerators

Some care must be taken when porting code to take advantage of HPC hardware:

- node-level performance should be understood and optimized before attempting parallelization (it is common that software that performs poorly at the node level seems to scale well on multiple nodes).
- Reaching good performance on GPU might require changes not only in the algorithms but also in the way data is stored in memory (the memory layout). In general, performance engineering is an integral part of porting software to another hardware architecture.

Moreover: notice that the available hardware typically changes every few years, and that the useful life of scientific software is typically longer. Maintenance costs can be reduced by making use of *performance portability frameworks*.

Performance Characterization of applications

Tools

There is a myriad of tools that can be used to study the performance of applications.

Taxonomy of performance analysis techniques and tools

Technique	Advantages, Disadvantages	Tools (examples)	Recommendation
total runtime/memory measurement	very simple, little information	<code>/usr/bin/time</code>	first thing to do
Profiling - Sampling	low overhead, imprecise results	Perf, gprof, Intel VTune, Julia's Profile	Best for initial assessment
Profiling (Instrumentation)	precise results, overhead needs to be managed	score-p/scalasca LIKWID (marker API) Python's cProfile gprof GPU profilers	choose region/functions to observe, estimate overhead from baseline
Simulation	Precision, deep understanding, extremely slow	valgrind/callgrind, machine code analysis	
Tracing	show task dependencies, HUGE output	Intel ITAC, Nvidia NSight, BSC Extrae, (Score-p)	create <i>mini app</i> with short run time

Walkthrough: Dense Matrix-Matrix multiplications (Part 1)

The Dense Matrix-Matrix multiplication has already very well optimized solutions, but we can learn something along the way by trying to optimize it, starting from the naive version.

Our code is computing the quantity

$$\backslash(D = B^T A B \backslash)$$

Baseline: Naive version, naive compilation

! Note

To follow along: commit [7a83cbd0a](#)

```
git checkout 7a83cbd0a
```

```
HEAD is now at 7a83cbd Matmul baseline
```

Compilation:

```
gcc -o main main.c -lm
```

Basic timing:

```
time ./main
```

```
real    0m24.341s
user    0m24.321s
sys     0m0.018s
```

The cpu is very busy. What is happening? In our code, we have some initialization functions, and a pair of matrix multiplications.

We expect the initialization functions to be negligible. Is this the case?

Let's have a look with `perf`. First of all, we compile with *debug symbols on*:

```
gcc -g -o main main.c -lm
```

Then we can run our code under `perf`:

```
perf record -g ./main
```

WARNING: Kernel address maps (/proc/{kallsyms,modules}) are restricted, check /proc/sys/kernel/kptr_restrict and /proc/sys/kernel/perf_event_paranoid.

Samples in kernel functions may not be resolved if a suitable vmlinux file is not found in the buildid cache or in the vmlinux path.

Samples in kernel modules won't be resolved at all.

If some relocation was applied (e.g. kexec) symbols may be misresolved even with a suitable vmlinux or kallsyms file.

Couldn't record kernel reference relocation symbol
Symbol resolution may be skewed if relocation was used (e.g. kexec).
Check /proc/kallsyms permission or run as root.
[perf record: Woken up 29 times to write data]
[perf record: Captured and wrote 7,423 MB perf.data (96993 samples)]

We can have a look at the recorded data:

```
# Using --stdio and head to stay on the notebook, you don't have to  
perf report -g --stdio | head -n 40
```

Warning:

Kernel address maps (/proc/{kallsyms,modules}) were restricted.

Check /proc/sys/kernel/kptr_restrict before running 'perf record'.

As no suitable kallsyms nor vmlinux was found, kernel samples can't be resolved.

Samples in kernel modules can't be resolved as well.

To display the perf.data header info, please use --header/--header-only options.

#

#

Total Lost Samples: 0

#

Samples: 96K of event 'cycles:P'

Event count (approx.): 103878286417

#

Children Self Command Shared Object Symbol

.....
#

99.99%	0.00%	main	libc.so.6	[.] __libc_start_call_main
--------	-------	------	-----------	----------------------------

|

---__libc_start_call_main

main

|

|--63.72%--mat_t_mat_mul

|

--36.09%--matmat_mul

99.99%	0.00%	main	main	[.] main
--------	-------	------	------	----------

|

---main

|

|--63.72%--mat_t_mat_mul

|

--36.09%--matmat_mul

63.72%	63.58%	main	main	[.] mat_t_mat_mul
--------	--------	------	------	-------------------

|

--63.58%--__libc_start_call_main

main

mat_t_mat_mul

36.09%	36.01%	main	main	[.] matmat_mul
--------	--------	------	------	----------------

|

--36.01%--__libc_start_call_main

main

matmat_mul

We note that:

1. The callgraph is very simple, so from now on we can avoid using `-g` with `perf`
2. the function `matmat_mul` (the one that does $C = AB$) takes half of the time as the `mat_t_mat_mul` (the one that does $D = B^T C$).

What is going wrong? We might have an idea about memory layouts and access patterns.

Investigation with the LIKWID Marker API

! Note

To follow along: [a9d5657ed](#)

```
git checkout a9d5657ed
```

Previous HEAD position was 7a83cbd Matmul baseline
HEAD is now at a9d5657 Minimal LIKWID marker instrumentation

We will use LIKWID to carry on the investigation. For this, we need to manually instrument the code:

```
git diff HEAD~
```

```
diff --git a/matmul/main.c b/matmul/main.c
index 7efd5ce..48cb3eb 100644
--- a/matmul/main.c
+++ b/matmul/main.c
@@ -1,6 +1,6 @@
#include "matmul_v1.c"
#include "init.c"
-
+#include <likwid-marker.h>

int main(){

@@ -14,10 +14,20 @@ int main(){
    // We want to compute D = Bt A B
    // C = A B
    // D = Bt C
+
+    LIKWID_MARKER_INIT;
+    LIKWID_MARKER_REGISTER("MM");
+    LIKWID_MARKER_REGISTER("MTM");
+
+    LIKWID_MARKER_START("MM");
+    matmat_mul(a,b,c,N);
+    LIKWID_MARKER_STOP("MM");
+    LIKWID_MARKER_START("MTM");
+    mat_t_mat_mul(b,c,d,N);
+    LIKWID_MARKER_STOP("MTM");

    free(d);free(c);free(b);free(a);
+    LIKWID_MARKER_CLOSE;
    return 0;

}
```

And then recompile:

```
gcc -DLIKWID_PERFMON -o main main.c -lm -llikwid
```

Once the compilation has succeeded (troubleshooting: see [this tutorial](#) if libraries and headers are not found), we can check which performance groups we can investigate with LIKWID:

```
likwid-perfctr -a
```

Group name	Description
ENERGY	Power and Energy consumption
FLOPS_DP	Double Precision MFLOP/s
DATA	Load to store ratio
DIVIDE	Divide unit information
FLOPS_SP	Single Precision MFLOP/s
BRANCH	Branch prediction miss rate/ratio
FLOPS_AVX	Packed AVX MFLOP/s

Since our code uses only `floats`, we expect the `FLOPS_SP` performance group to be interesting:

```
likwid-perfctr -m -C S0:0 -g FLOPS_SP ./main
```

CPU name: 11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz
CPU type: Intel Tigerlake processor
CPU clock: 2.61 GHz

Region MM, Group 1: FLOPS_SP

+-----+-----+
| Region Info | HWThread 0 |
+-----+-----+
| RDTSC Runtime [s] | 9.417304 |
| call count | 1 |
+-----+-----+

+-----+-----+-----+
| Event | Counter | HWThread 0 |
+-----+-----+-----+
INSTR_RETIRED_ANY	FIXC0	104535300000
CPU_CLK_UNHALTED_CORE	FIXC1	37669540000
CPU_CLK_UNHALTED_REF	FIXC2	24342280000
FP_ARITH_INST_RETIRED_128B_PACKED_SINGLE	PMC0	0
FP_ARITH_INST_RETIRED_SCALAR_SINGLE	PMC1	2985984000
FP_ARITH_INST_RETIRED_256B_PACKED_SINGLE	PMC2	0
FP_ARITH_INST_RETIRED_512B_PACKED_SINGLE	PMC3	0
+-----+-----+-----+

+-----+-----+
| Metric | HWThread 0 |
+-----+-----+
Runtime (RDTSC) [s]	9.4173
Runtime unhalted [s]	14.4266
Clock [MHz]	4040.6872
CPI	0.3604
SP [MFLOP/s]	317.0742
AVX SP [MFLOP/s]	0
AVX512 SP [MFLOP/s]	0
Packed [MUOPS/s]	0
Scalar [MUOPS/s]	317.0742
Vectorization ratio [%]	0
+-----+-----+

Region MTM, Group 1: FLOPS_SP

+-----+-----+
| Region Info | HWThread 0 |
+-----+-----+
| RDTSC Runtime [s] | 17.361890 |
| call count | 1 |
+-----+-----+

+-----+-----+-----+
| Event | Counter | HWThread 0 |
+-----+-----+-----+
INSTR_RETIRED_ANY	FIXC0	104541300000
CPU_CLK_UNHALTED_CORE	FIXC1	72267170000
CPU_CLK_UNHALTED_REF	FIXC2	44929640000
FP_ARITH_INST_RETIRED_128B_PACKED_SINGLE	PMC0	0
FP_ARITH_INST_RETIRED_SCALAR_SINGLE	PMC1	2985984000
FP_ARITH_INST_RETIRED_256B_PACKED_SINGLE	PMC2	0
FP_ARITH_INST_RETIRED_512B_PACKED_SINGLE	PMC3	0
+-----+-----+-----+

+-----+-----+
| Metric | HWThread 0 |
+-----+-----+

+-----+		
	Runtime (RDTSC) [s]	17.3619
	Runtime unhaltd [s]	27.6767
	Clock [MHz]	4199.8549
	CPI	0.6913
	SP [MFLOP/s]	171.9850
	AVX SP [MFLOP/s]	0
	AVX512 SP [MFLOP/s]	0
	Packed [MUOPS/s]	0
	Scalar [MUOPS/s]	171.9850
	Vectorization ratio [%]	0
+-----+		

An observation: the two functions do the same amount of operations, but for the matrix-time-matrix multiplication we have a ~70% better performance than for the matrix-transpose-time-matrix multiplication.

This already suggests that at least one of the two function could be written in a way that is far from optimal.

Before we go further, we should ask ourselves:

1. How far are we from the maximum performance?
2. How much more performance would we be able to reach?

Elements of Computer Architecture and their effects

Compute balance: Raw compute power is cheap, memory bandwidth is not. This is why most HPC workloads are memory bound.

Tools for system investigation

To understand what are the limits of the system we are working on, we can use some predefined benchmarks to assess the performance of the system.

- [likwid-topology](#)
- [likwid-bench](#): suite of tunable micro benchmarks for CPU and main memory
- [fio](#) for IO benchmarks and tests
- [OSU Micro Benchmarks](#): suite of micro benchmarks for different types of MPI communication

Trade-offs

Demo: Hardware exploration with LIKWID

What is the structure of the CPU? `likwid-topology` can help us. We can use `likwid-bench` to explore the limits of our hardware.

For an exploration, see [this notebook](#)

- Memory: Size vs Speed and latency.

Characteristics of memory systems (Order of magnitude, from various sources)

System	Bandwidth	Latency	Size	Notes
Registers	*	< 1cy	1KB	per core
L1 cache	<100GB/s	~1ns	32KB	per core
L2 cache	<100GB/s	~4ns	512KB	per core
L3 cache	1.25 TB/s	~16ns	50MB	per package, shared
HBM	2-8TB/s		64GB	per package
Main Memory	50-400GB/s	~100ns(*)	200GB	per node
Network (IB)	<100GB/s		-	
Local SDD	~1GB/s		~1TB	
Global Filesystem	~50GB/s		~PBs	
Tape		~1min		

With `likwid-topology` and `likwid-bench` we can get more precise values or better estimates.

Challenge: What if our data structures do not fit in memory/storage system X?

- Memory granularity:
 - Registers (there's only a handful of them unfortunately)
 - Cache line (64B on most CPUs, 64B-128B on GPUs)
 - Memory Pages: >4MB

Parallelism in the core: Vectorization and superscalarity

CPUs can do more things at the same time in two main ways:

- Vectorization (SIMD, Single Instruction Multiple Data):
- Superscalarity and pipelining

Vector operations

Note: Vector loads/store have typically higher bandwidth, especially when fetching data from L1 (verify with `likwid-bench`).

Superscalarity

Caches

- Data is moved between cache levels and between caches and the RAM only in “chunks” of 64Bytes (the size of a *cache line*).

This means that if we use only a single value out of a 64B cache line, we will potentially trash a lot of data.

- Cache is typically not fully associative: not all 64B chunks of the main memory can be “mirrored” in all cache lines, but typically each of them can only fit in a specific “cache line group”.

Some access patterns can lead to poor cache utilization due to **cache associativity**.

- Cache levels can also be inclusive, exclusive or neither ([cache inclusion policy](#)).

False sharing

What is the best way to split embarrassingly parallel work on an array by N threads?

1. Thread 0 works on element 0, Thread 1 works on element 1 and in general Thread `i` works on elements `k*nthreads+i` ;
2. Array is split in `nthreads` contiguous chunks, every thread works on a chunk.

✓ Solution

When working with multiple threads, be aware that different threads working on different cores accessing the same data might “fight” over a single cache line, as the hardware is forced to give a consistent view of the data to both threads. In that situation, called *false sharing*, the workload becomes latency-bound. For this reason, typically the best approach is 2.

Note that this is exactly the opposite to what is done when writing GPU kernels, where each thread in a thread block works typically on data that is close to the data on which the other members of the thread bloc operate: we are operating at another granularity level.

Main memory

Rows and columns: memory is usually accessed in rows and columns. If different data is read from the same row repeatedly, then access is typically noticeably faster. This is another reason why sequential access can be much faster than random access.

Non-Uniform-Memory-Access

On modern and powerful compute nodes it is very common nowadays the situation where not all compute units can access the main memory in an *uniform* way.

Bandwidth and Latency

Situation	Bandwidth	Latency
Same domain	165GB/s	90 ns
Different domain	55GB/s	140 ns

Simple performance modelling: the roofline model

Recommendations

- Ensuring Data reuse is paramount:
 - use as much data as possible out of every cache line before it is evicted (split “hot” and “cold” data between different structures/arrays)
 - reuse as much as possible data that has been read or written shortly before

References

- [Infiniband max Bandwidth](#)
- [Algorithmica: Algorithms for Modern Hardware](#)
- [What every programmer should know about memory](#)

A brief LIKWID demo: topology and microbenchmarks

LIKWID is a suite of tools for performance diagnostics. In this short demo we will use two tools to inspect the property of the hardware, `likwid-topology` and `likwid-bench`

Inspect node structure with `likwid-topology`

`likwid-topology` aggregates information coming from different sources and reports on the topology of the CPU, its caches and NUMA domains (NVidia GPU information can be obtained with -G).

Basic information can be obtained with:

```
likwid-topology --clock --caches
```

CPU name: 11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz
CPU type: Intel Tigerlake processor
CPU stepping: 1
CPU clock: 2.61 GHz

Hardware Thread Topology

Sockets: 1
CPU dies: 1
Cores per socket: 4
Threads per core: 2

HWThread	Thread	Core	Die	Socket	Available
0	0	0	0	0	*
1	0	1	0	0	*
2	0	2	0	0	*
3	0	3	0	0	*
4	1	0	0	0	*
5	1	1	0	0	*
6	1	2	0	0	*
7	1	3	0	0	*

Socket 0: (0 4 1 5 2 6 3 7)

Cache Topology

Level: 1
Size: 48 kB
Type: Data cache
Associativity: 12
Number of sets: 64
Cache line size: 64
Cache type: Non Inclusive
Shared by threads: 2
Cache groups: (0 4) (1 5) (2 6) (3 7)

Level: 2
Size: 1.25 MB
Type: Unified cache
Associativity: 20
Number of sets: 1024
Cache line size: 64
Cache type: Non Inclusive
Shared by threads: 2
Cache groups: (0 4) (1 5) (2 6) (3 7)

Level: 3
Size: 8 MB
Type: Unified cache
Associativity: 8
Number of sets: 16384
Cache line size: 64
Cache type: Non Inclusive
Shared by threads: 8
Cache groups: (0 4 1 5 2 6 3 7)

NUMA Topology

NUMA domains: 1

```

Domain:                0
Processors:             ( 0 4 1 5 2 6 3 7 )
Distances:              10
Free memory:            1738.41 MB
Total memory:           15593.6 MB
-----

```

Notably, we get:

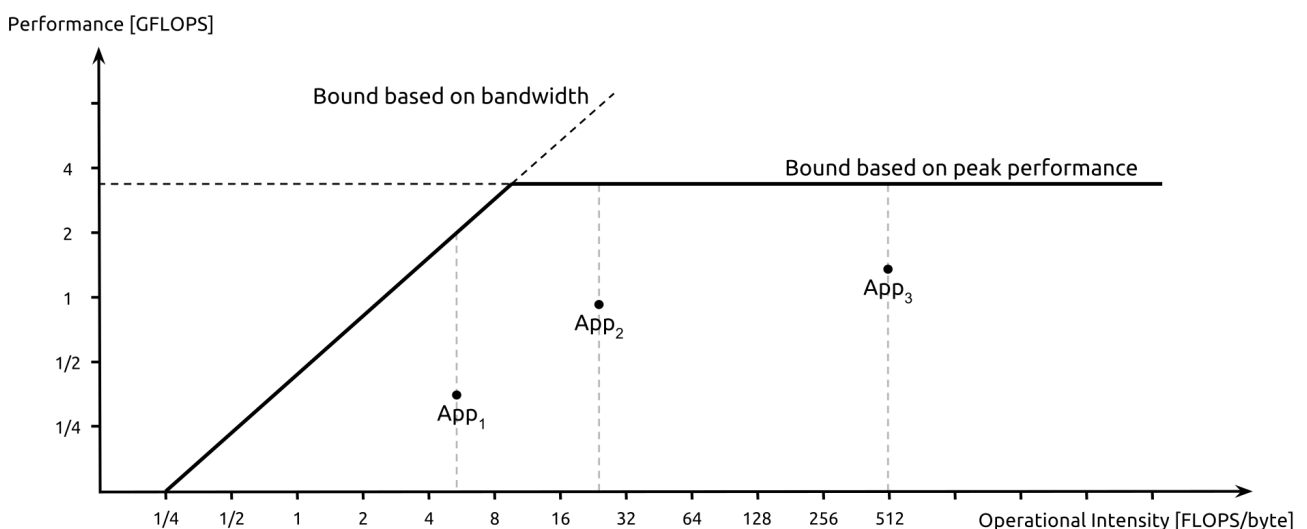
- basic cpu and threading information (thread to core correspondence)
- **cache information**: in this case, we see that L1 and L2 are local to a core (which corresponds to a pair of threads), while L3 cache is common to all cores and threads. We also get the sizes of the caches (which we should be able to compare with some experiments, later)
- **NUMA topology** information: in this case, there is only one domain to which all processors belong.

Maximum performance and bandwidth estimates: the Roofline model

The [Roofline Model](#) is a performance model that assumes that there are 2 main performance bottlenecks:

- peak floating point performance
- the memory bandwidth.

In this model, the relevant bottleneck for the kernel (typically, a loop) being investigated is determined by its **arithmetic intensity** (also **operational intensity**), that is the ratio between how many FP operations are done and the *memory traffic* in byte, that is how many bytes have to be moved from/to a particular memory system (cache level or RAM).



By [Giu.natale](#) - Own work, [CC BY-SA 4.0](#), [Link](#)

The maximum possible FP performance for a kernel is given by that “roofline” (called so probably because it reminds of a roof):

- kernels which require few operations per byte (low arithmetic intensity, on the left side of the plot above) will be ultimately **memory bound**
- kernels which perform enough operations per byte (right end of the plot above) will be **compute bound**, that is able to run at the maximum FP performance of the machine.

Most codes in use today are typically **memory bound**.

Notice that if our code is *latency limited* its performance will stay way below the roofline.

likwid-bench

LIKWID gives us a way to determine the characteristics of our machine using `likwid-bench`, a suite of microkernel benchmarks.

In order to use `likwid-bench` we need to specify, at least:

- The “thread domain”, that is on which set of hardware threads the benchmarks is going to run.

To get which thread domains are available, we can get the `-p` option:

```
likwid-bench -p
```

```
Number of Domains 5
Domain 0:
    Tag N: 0 4 1 5 2 6 3 7
Domain 1:
    Tag S0: 0 4 1 5 2 6 3 7
Domain 2:
    Tag D0: 0 4 1 5 2 6 3 7
Domain 3:
    Tag C0: 0 4 1 5 2 6 3 7
Domain 4:
    Tag M0: 0 4 1 5 2 6 3 7
```

On a more complicated machine than my laptop we typically get a `N` domain for the whole node, and one per socket or NUMA domain, and one per shared cache.

- The working set size, that is how big the dataset for the benchmark is going to be. This determines mostly *which cache levels will be involved*: bigger dataset will need larger and slower caches.

By tuning the working set size and comparing it with the cache sizes we get from `likwid-topology` we can get an idea the properties of the different cache levels.

- The benchmark type we want to run. A (long) list of the readily available benchmarks (for the architecture of the machine we are running on) can be obtained with `likwid-bench -a`. To understand the “maximum” performance and bandwidth of the machine, we are interested in the `peakflops_*` and `load_*` microkernels:

```
likwid-bench -a | grep 'peakflops_\|load_' # using Grep to limit output for readability
```

```
load_avx - Double-precision load, optimized for AVX
load_avx512 - Double-precision load, optimized for AVX-512
load_mem - Double-precision load, using non-temporal loads
load_sse - Double-precision load, optimized for SSE
peakflops_avx - Double-precision multiplications and additions with a single load,
optimized for AVX
peakflops_avx512 - Double-precision multiplications and additions with a single load,
optimized for AVX-512
peakflops_avx512_fma - Double-precision multiplications and additions with a single
load, optimized for AVX-512 FMAs
peakflops_avx_fma - Double-precision multiplications and additions with a single load,
optimized for AVX FMAs
peakflops_sp - Single-precision multiplications and additions with a single load, only
scalar operations
peakflops_sp_avx - Single-precision multiplications and additions with a single load,
optimized for AVX
peakflops_sp_avx512 - Single-precision multiplications and additions with a single
load, optimized for AVX-512
peakflops_sp_avx512_fma - Single-precision multiplications and additions with a single
load, optimized for AVX-512 FMAs
peakflops_sp_avx_fma - Single-precision multiplications and additions with a single
load, optimized for AVX FMAs
peakflops_sp_sse - Single-precision multiplications and additions with a single load,
optimised for SSE
peakflops_sse - Double-precision multiplications and additions with a single load,
optimised for SSE
```

Determining the maximum performance

The benchmarks named `peakflops_*` can give us an estimate of the maximum FLOPs performance of our machine.

In order to run these benchmarks, we choose a small dataset that is going to fit in the L1 CPU cache (otherwise the performance might be limited by memory bandwidth and not by the actual FP peak performance), and choose the whole machine as thread domain (`N`). As shown before by `likwid-topology`, the Total L1 cache for my CPU is 4*48\$KB, so I will choose 128KB as the size of the working set.

A few comments:

- By not specifying a number of threads, `likwid-bench` will use all the hardware threads in the workgroup by default (here, 8)
- We are using all 4 cores on my machine

- maximum performance for *scalar*, double-precision operations:

```
likwid-bench -t peakflops -W N:128KB | grep 'MFlops/s' # Using grep here to focus only on the most relevant output
```

```
Running without Marker API. Activate Marker API with -m on commandline.  
MFlops/s: 30087.87
```

- maximum performance for *vector* (256-bit), double-precision operations:

```
likwid-bench -t peakflops_avx -W N:128KB | grep 'MFlops/s' # Using grep here to focus only on the most relevant output
```

```
Running without Marker API. Activate Marker API with -m on commandline.  
MFlops/s: 117079.56
```

- maximum performance for *vector* (256bit), double precision operations, which *fused multiply-add* operations:

```
likwid-bench -t peakflops_avx_fma -W N:128KB | grep 'MFlops/s' # Using grep here to focus only on the most relevant output
```

```
Running without Marker API. Activate Marker API with -m on commandline.  
MFlops/s: 232018.68
```

Now, comparing with the corresponding single precision operations:

```
likwid-bench -t peakflops_sp -W N:128KB | grep 'MFlops/s' # scalar  
likwid-bench -t peakflops_sp_avx -W N:128KB | grep 'MFlops/s' # vector  
likwid-bench -t peakflops_sp_avx_fma -W N:128KB | grep 'MFlops/s' # vector + FMA
```

```
Running without Marker API. Activate Marker API with -m on commandline.  
MFlops/s: 30032.48  
Running without Marker API. Activate Marker API with -m on commandline.  
MFlops/s: 210624.63  
Running without Marker API. Activate Marker API with -m on commandline.  
MFlops/s: 373403.05
```

A few comments:

- Using vector operations (256b) lead ~4 times more performance than scalar ones (for double precision, 8 for single precision)
- The 'peak value' depends on the kind of operation.

Determining the maximum bandwidth

We can use now the `load_*` benchmarks to find out the memory bandwidth. Depending on the size of the working set, we can also evaluate the bandwidth for each cache level.

With a working set of 2GB we will for sure hit the RAM:

```
likwid-bench -t load -W N:2GB | grep 'MByte/s'
likwid-bench -t load_avx -W N:2GB | grep 'MByte/s'
```

```
Running without Marker API. Activate Marker API with -m on commandline.
MByte/s:                43969.41
Running without Marker API. Activate Marker API with -m on commandline.
MByte/s:                43688.51
```

With a working set smaller than the L3 cache, we should hit mostly the L3 cache, and thus our bandwidth should be quite higher:

```
likwid-bench -t load -W N:6.4MB | grep 'MByte/s'
likwid-bench -t load_avx -W N:6.4MB | grep 'MByte/s'
```

```
Running without Marker API. Activate Marker API with -m on commandline.
MByte/s:                114149.95
Running without Marker API. Activate Marker API with -m on commandline.
MByte/s:                190416.17
```

If we want to measure the bandwidth of the L2 cache, we can use a working set that is smaller than the total L2 cache, which is 4×1.25 MB:

```
likwid-bench -t load -W N:4MB | grep 'MByte/s'
likwid-bench -t load_avx -W N:4MB | grep 'MByte/s'
```

```
Running without Marker API. Activate Marker API with -m on commandline.
MByte/s:                125898.96
Running without Marker API. Activate Marker API with -m on commandline.
MByte/s:                317482.83
```

Fitting now in the L1 cache:

```
likwid-bench -t load -W N:168KB | grep 'MByte/s'
```

Running without Marker API. Activate Marker API with -m on commandline.
MByte/s: 178798.50

From the L1 cache we should also see a *huge* difference when using vector load instructions:

```
likwid-bench -t load_avx -W N:168KB | grep 'MByte/s'
```

Running without Marker API. Activate Marker API with -m on commandline.
MByte/s: 729787.97

What about stores?

```
likwid-bench -t store -W N:2GB | grep 'MByte/s'
```

Running without Marker API. Activate Marker API with -m on commandline.
MByte/s: 15096.93

Shortcut: Bypassing the cache with non-temporal stores

If some data must be written to a memory location that we know will not be accessed again in the near future (that means, it does not make sense to cache it), we can use *non-temporal stores*, which are typically faster. The benchmark `store_mem` uses non-temporal stores:

```
likwid-bench -t store_mem -W N:2GB | grep 'MByte/s'
```

Running without Marker API. Activate Marker API with -m on commandline.
MByte/s: 37493.32

Appendix

More about `likwid-topology`

As usual, to get information about `likwid-topology` we can use the `-h` option:

```
likwid-topology -h
```

```
likwid-topology -- Version 5.4.1 (commit: 0123456789)
A tool to print the thread and cache topology on CPUs and GPUs.
```

Options:

-h, --help	Help message
-v, --version	Version information
-V, --verbose <level>	Set verbosity
-c, --caches	List cache information
-C, --clock	Measure processor clock
-G, --gpus	List GPU information
-O	CSV output
-o, --output <file>	Store output to file. (Optional: Apply text filter)
-g	Graphical output

More about `likwid-bench`

We can get more information with the `-h` option:

```
likwid-bench -h
```

Supported Options:

```
-h/--help           Help message
-v/--version        Version
-a/--all            List available benchmarks
-d/--delim          Delimiter used for physical hwthread list (default ,)
-p/--printdomains   List available thread domains
                   or the physical ids of the hwthreads selected by the -
c expression
-s/--runtime <TIME> Seconds to run the test minimally (default 1)
                   If resulting iteration count is below 10, it is
normalized to 10.
-i/--iters <ITERS>  Specify the number of iterations per thread manually.
-l/--list <TEST>     list properties of benchmark
-t/--test <TEST>     type of test
-w/--workgroup       <thread_domain>:<size>[:<num_threads>[:<chunk size>:<stride>]]-
<streamId>:<domain_id>[:<offset>]
-W/--Workgroup       <thread_domain>:<size>[:<num_threads>[:<chunk size>:<stride>]]
                   <size> in kB, MB or GB (mandatory)

For dynamically loaded benchmarks
-f/--tempdir <PATH>  Specify a folder for the temporary files. default: /tmp
-o/--asmout <FILE>   Save generated assembly to file
```

Difference between -w and -W :

-w allocates the streams in the thread_domain with one thread and support placement of streams

-W allocates the streams chunk-wise by each thread in the thread_domain

Usage:

```
# Run the store benchmark on all CPUs of the system with a vector size of 1 GB
likwid-bench -t store -w S0:1GB
# Run the copy benchmark on one CPU at CPU socket 0 with a vector size of 100kB
likwid-bench -t copy -w S0:100kB:1
# Run the copy benchmark on one CPU at CPU socket 0 with a vector size of 100MB but
place one stream on CPU socket 1
likwid-bench -t copy -w S0:100MB:1-0:S0,1:S1
```

See also the official [wiki page](#) for less terse explanations.

Other tools

A “simple” way to measure latency is also using “pointer chasing” (see [this optional episode](#) for an example).

The [Intel mlc \(Memory Latency Checker\)](#) can not only measure latencies, but also determine the latency-bandwidth curve (when the bandwidth is saturated, then latency tends to increase too).

Credits

Apart from personal experience, the content of this notebook is inspired by official [LIKWID wiki](#), and in particular the [tutorial on the empirical roofline model](#).

What a compiler (and the hardware) can do for you

Compiler as magicians

With the right flags, compilers can do automatically:

- common subexpression elimination
- Loop unrolling
- Loop reordering
- Auto vectorization
- Some compilers (i.e., Intel) also can auto-parallelize loops using shared-memory parallelization.

Note: Some advanced optimization options can actually change subtly the actual behaviour of the code.

Common optimizations enabled via flags

- Generation of machine instructions for a specific architecture.
- `-Olevel`: Different optimization levels “meta” options, typically corresponding to a set of flags for specific optimizations. Higher levels typically include all the optimization flags included lower levels, plus more.

Higher optimization levels tend to:

- offer higher performance binaries (even by orders of magnitude),
- increase the compilation times.
- remove checks and debugging information

For this reason, consider:

- For testing during your usual development loop, use `-O0`
- For debugging, use `-Og` (if available)
- For performance: use `-O2` / `-O3` or `-Ofast`

Compilation options: RTFM

Familiarize yourself with the manual of the GNU and Intel compiler, with

```
man gcc
```

for the GNU C compiler, and

```
man icx
```

for the Intel C compiler.

If you are doing this on a HPC system, you might need to load the appropriate modules to make the manpages for the compilers available.

What are the default optimization options in both cases?

Note: different compilers have different default values of `-Olevel`, some defaulting to `-O0`, some to `-O2`.

- “fast math” optimization options: these allow the compiler to use versions of the math functions that do not have to comply to standard, and thus could be faster.

How to help a compiler

There are some modifications that might help a compiler produce better assembly code.

- Avoid pointer hopping and indirect addressing (this requirement, in higher level languages, also translates in avoiding dynamic typing in some situations)
- Loop unrolling (this can be done automatically by modern compilers)

Automatic vectorization

Vectorized instructions represent one of the most important ways to get high performance from modern CPUs.

- Use an appropriate memory layout. Going from an *Array of Structures* (AoS) to a *Structure of Array* (SoA) layout might help. Reads and writes must be done in 128/256/512 bit chunks.
- Avoid complex logic in loops that could be vectorized, in particular:
 - conditionals (e.g., `if` statements)
 - calls to non-trivial functions that cannot be inlined.

To verify the result of the vectorization, there are a few options

1. Get the assembly output either by
 - running `objdump -d` on the object file produced (can work also with the final linked binary, but might be way larger)
 - compile the source file with the `-S` option
2. Get the information from the compiler in the form of a vectorization report (`-fopt-info-vec` for GNU, `-qopt-report=max` for Intel's icx/icxx compiler).

The role of hardware: Pipelining (and stalls)

Once the compiler has produced the assembly instructions, the actual way they are executed depends on the hardware.

After being fetched from memory, each assembly instruction still typically to be *decoded* potentially into a number of more fundamental instructions (μ OPS, micro-ops)¹. Most assembly instructions on a x86 architecture do take more than one cycle to complete.

Since in order to fetch, decode and execute an instruction different elements of the CPU core are needed at different stages of the processing, in order to keep the whole CPU core busy all the time multiple instructions are processed at the same time but in different stages, in a *pipeline* fashion ². This approach is able to greatly increase throughput by “masking” the latency associated to the processing of a single instruction.

Also, different instructions or μ OPS can be executed at the same time and even *out-of-order*, thanks to the *superscalar architecture* of modern CPUs cores in which some processing unit are present in multiple copies.

Pipeline stalls

Pipelines can get stalled in different ways, for example:

- when waiting for data to arrive. In that case, typically simultaneous multithreading (*hyperthreading*) can be used efficiently to mask these latencies
- in a conditional statement, the evaluation of the condition might delay the execution of the dependent instructions. The hardware typically tries to compensate with branch prediction and speculative execution.

[1] This is typical of x86 architectures, while on ARM the decoding is typically simpler.

[2] See for example [this explanation](#) in the RISC case.

Reproducibility problems in performance measurements

Consistent results in measurements are paramount to properly inform judgement and action when doing performance optimization.

Common sources of irreproducibility

If you repeat a performance measurement and you notice an unexpected variation, consider these aspects:

Non-comprehensive list of sources of irreproducibility, and their mitigations

Class	Cause	Mitigation/Solution	Notes
Software	Software versions (dependencies)	Proper dependency tracking, with pinned (<code>.lock</code> files, <code>Manifest.toml</code>) under version control	
	Compiler flags	Automation of builds, build scripts under VC	(also for Julia)
Hardware	CPU frequency variations	<code>likwid-setFrequencies</code> , <code>--cpu-freq</code> SLURM flag, system monitoring (<code>CC</code> , <code>JM</code>), <code>MachineState</code>	factor of 4 observed
	Microcode updates	Track microcode versions (<code>cat /proc/cpuinfo</code>)	(Rare)
Multithreading	Thread migration (process migration)	thread/process pinning with OMP/MPI (<code>likwid-pin</code> , <code>ThreadPinning.jl</code>)	
	Dynamic thread scheduling	Use static scheduling instead (if reasonable)	dynamic scheduling necessary for load balancing (at times)
“Noisy Neighbour”	node sharing	use <code>--exclusive</code> <code>sbatch</code> allocation	
	shared filesystem congestion	Isolate, manage and monitor I/O	
	Network congestion	topology control via <code>sbatch</code>	

Credits

The current course material is inspired by and based on:

- The course [Introduction to Performance Engineering on HPC](#) by Holger Obermaier and Begatim Bytyqi from KIT
- The material for [Node Level Performance Engineering](#) course at HLRS
- The [Algorithmica.org](#) book/website by Sergej Slotin

Pointer Chasing and latency

It is possible to estimate the latency associated to different cache levels and to the main memory by implementing a pathological pattern of memory access where the hardware cannot at all predict what address will be accessed before the previous address has been loaded and read.

This problem, which affects also real-life codes, is called *pointer chasing* and it is typical of code where objects contain references (pointers) to other objects, which contain references to other objects and so on.

We will see this with a Julia example, based on [these notes](#).

Pointer chasing

This function will jump between positions in an array, based on addresses that are read from it:

```
function pointer_chasing(array_of_pointers::Vector{Int64})
    k = 1
    for _ in 1:size(array_of_pointers, 1)
        k = array_of_pointers[k]
    end
end
```

A possible way to create such an array so that *all* the locations are actually visited is:

```
function get_self_referential_array_random_order(array_size::Int64)
    all_indices = collect(1:array_size)
    single_cycle_array_visiting_order = shuffle(all_indices)
    res = similar(single_cycle_array_visiting_order)

    k = single_cycle_array_visiting_order[end]
    for i in eachindex(single_cycle_array_visiting_order)
        res[k] = single_cycle_array_visiting_order[i]
        k = res[k]
    end
    return res
end
```

A quick experiment: on my laptop, hopping through a 16MB array (enough to be on main memory) in an ordered way requires 1.2ns of latency per access on average, while doing the same in a totally random order requires 69ns of latency per access on average.

A simple latency model

The average latency will be a mean of the latency associated to the different cache levels, weighted by the size of the part of the dataset in each cache level.

A small enough array will fill in the L1 cache, and the resulting latency will be “pure” from L1. For array that “spills” into L2, the latency will be a combination of L1 and L2 contributions, and so on:

```

function latency_model(array_size::Int64, params)
    # We do not care about the side of the dram, but about its latency
    # We also assume that the caches are exclusive
    levels = Int((length(params) - 1) / 2)
    sizes = params[1:levels]
    cumsums = cumsum(cat([0], sizes, dims=1))
    latencies = params[levels+1:end]

    cache_contribution = sum(latency * min(size, array_size - cumsize) / array_size
        for (size, cumsize, latency)
        in
            zip(sizes, cumsums, latencies) if (array_size > cumsize))
    dram_contribution = if (array_size > sum(sizes))
        (array_size - sum(sizes)) / array_size * latencies[end]
    else
        0
    end

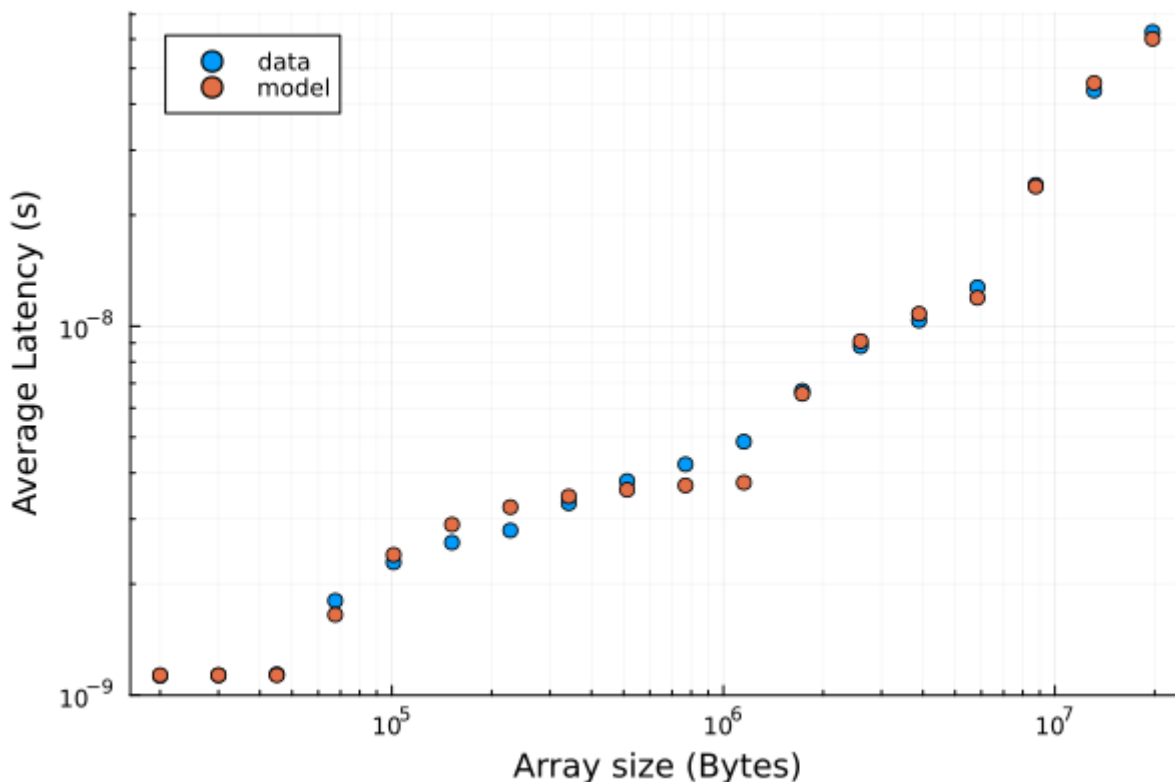
    return cache_contribution + dram_contribution
end

```

The parameters of the model are:

- the cache sizes
- the latency of all the cache levels, and the main memory latency

Fitting the latency model to the measured latencies requires a some care (enough data points, sensible initial values for the parameters, sensible form for the residuals)



The output of the [full julia script](#):

```
Fit results (assuming 3 cache levels + dram):
```

```
size: 54 KiB latency: 1 ns  
size: 1186 KiB latency: 4 ns  
size: 6034 KiB latency: 14 ns  
Main memory latency: 89.08 ns
```

The times have been measured with `@btime` .

The initial values from the cache sizes sizes (48KiB, 1.25MiB and 8 MiB) are actually from `likwid-topology` , so the deviation from the values resulting from the fit give an idea of the systematic error in the procedure.

By fixing the cache sizes to the initial, “official” values, we get:

```
Fit results (assuming 3 cache levels + dram):
```

```
size: 48 KiB latency: 1 ns  
size: 1282 KiB latency: 4 ns  
size: 8192 KiB latency: 16 ns  
Main memory latency: 116.02 ns
```